Fundamental 1팀 스터디 / 딥 러닝 파이토치 교과서

16 시퀀스투시퀀스 (seq2seq)

문장을 넣으면 다른 문장이 나오는 모델입니다. 입력 문장을 읽는 RNN과 출력 문장을 쓰는 RNN을 이어 붙여 영어를 프랑스어로 번역하는 모델을 만들고, 번역이 얼마나 잘됐는지 점수(BLEU)로 재어 봅니다.

오늘 다루는 것

1. 시퀀스투시퀀스: 인코더, 디코더, 컨텍스트 벡터
2. seq2seq를 이용한 번역기 구현하기
3. BLEU Score: 번역의 품질을 재는 방법

끝나고 나면 할 수 있는 것

1. 인코더와 디코더가 무엇을 주고받는지, 훈련할 때와 번역할 때 디코더가 어떻게 다르게 동작하는지 설명할 수 있다.
2. 번역 모델의 입력 두 개와 정답 하나가 각각 무엇인지 말할 수 있다.
3. BLEU가 무엇을 세어 점수를 내는지, 어떤 한계가 있는지 설명할 수 있다.

2026년 10월 14일 수요일 14:30 (10/14 자료 1/2)
실습은 모두 코랩 기본 환경에서 실행됩니다. 실습 2 → 3 → 5 순서로 실행합니다(실습 3은 CPU로 약 15분).

16

시퀀스투시퀀스 (Sequence-to-Sequence)

13장은 문장에 정답 하나를, 14장은 단어마다 정답 하나씩을 붙였습니다. 번역은 다릅니다. "I am a student"(4단어)의 답은 "je suis étudiant"(3단어)입니다. 입력과 출력의 길이가 다르고, 단어가 일대일로 대응하지도 않습니다. 이런 문제를 푸는 구조가 시퀀스투시퀀스(seq2seq)이고, 번역, 요약, 챗봇이 모두 이 틀로 만들어집니다.

왜 배우나: 오늘의 학습 경로



문장을 넣어 문장을 얻고 싶다
번역, 요약, 챗봇

입력 문장을 벡터로 요약한다
인코더, 컨텍스트 벡터

그 벡터에서 한 단어씩 만든다
디코더

정답을 보여 주며 훈련한다
교사 강요

번역하고 점수를 매긴다
번역기, BLEU

장 전체의 상황은 영어를 프랑스어로 번역하는 일입니다. 16-01에서는 "I am a student → je suis étudiant" 한 문장으로 구조를 보고, 16-02에서는 문장 쌍 33,000개로 실제 번역기를 학습시키며, 16-03에서 그 번역기에 점수를 매깁니다. 18장(트랜스포머)에서 쓴 인코더, 디코더, 교사 강요가 처음 나오는 곳이 여기입니다.

16-01 시퀀스투시퀀스

핵심 한 줄

seq2seq는 인코더 RNN이 입력 문장을 읽어 컨텍스트 벡터 하나로 요약하고, 디코더 RNN이 그 벡터에서 출발해 <eos>가 나올 때까지 단어를 하나씩 만드는 구조다.

왜 배우나

새 부품은 없습니다. 임베딩, LSTM, 선형층, 소프트맥스는 모두 배운 것입니다. 새로운 것은 **조립 방법**입니다. RNN 두 개를 어떻게 잇는지, 훈련할 때와 실제로 쓸 때 디코더가 왜 다르게 동작하는지를 이해하면 16-02의 코드는 그 순서대로 읽힙니다.

1) 인코더, 디코더, 컨텍스트 벡터

정의

- 시퀀스투시퀀스(seq2seq): 입력 시퀀스를 받아 다른 시퀀스를 출력하는 모델. 두 시퀀스의 길이는 달라도 됩니다.
- 인코더(encoder): 입력 문장의 단어를 차례로 읽는 RNN. 다 읽은 뒤의 **마지막 은닉 상태**만 내보냅니다.
- 컨텍스트 벡터(context vector): 인코더의 마지막 은닉 상태. 입력 문장 전체를 요약한 벡터 하나입니다. LSTM이라면 은닉 상태와 셀 상태 두 개를 함께 넘깁니다.
- 디코더(decoder): 출력 문장을 만드는 RNN. 컨텍스트 벡터를 **자신의 첫 은닉 상태**로 삼아 시작합니다. 시점마다 "다음에 올 단어"를 예측합니다. 하는 일은 10장의 언어 모델과 같고, 입력 문장의 요약에서 출발한다는 점만 다릅니다.
- `<sos>` (start of sequence), `<eos>` (end of sequence): 문장의 시작과 끝을 알리는 특별 토큰. 디코더는 <sos>를 받아 시작하고, <eos>를 내면 멈춥니다.

디코더가 단어를 고르는 방법. 시점마다 디코더 LSTM의 출력을 선형층에 넣어 출력 언어의 단어 집합 크기만큼의 점수를 얻고, 소프트맥스로 확률로 바꿉니다(05장). 시점마다 "단어 집합 중 하나를 고르는 분류 문제"를 푸는 것입니다.

왜 RNN을 두 개로 나눌까? 13장처럼 RNN 하나의 마지막 출력만 쓰면 답이 하나뿐이고, 14장처럼 시점마다 답을 내면 출력의 길이가 입력과 같아야 합니다. 읽는 일과 쓰는 일을 나누면, 디코더는 입력의 길이와 상관없이 <eos>가 나올 때까지 필요한 만큼 단어를 낼 수 있습니다.

예시 적용

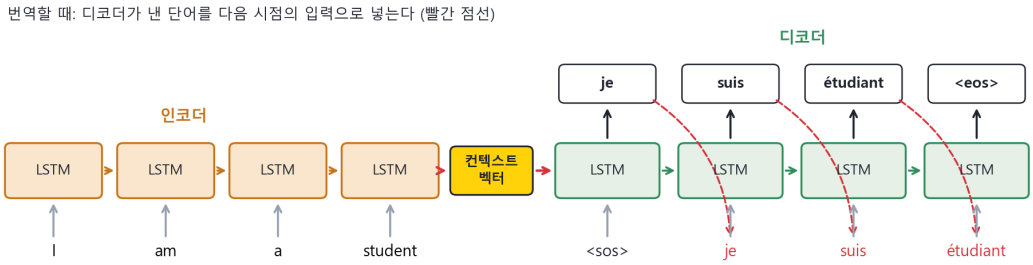


그림 1 "I am a student"를 번역하는 seq2seq. 인코더(주황)가 넘긴 컨텍스트 벡터에서 디코더(초록)가 시작한다

1. 인코더가 I, am, a, student를 차례로 읽습니다. 마지막 은닉 상태가 컨텍스트 벡터입니다.
2. 디코더가 컨텍스트 벡터와 <sos>를 받아 첫 단어 je를 냅니다.
3. 방금 낸 je를 다음 시점의 입력으로 넣어 suis를 냅니다. 같은 방식으로 étudiant를 냅니다.

4. étudiant를 넣으면 <eos>가 나오고, 번역이 끝납니다.

2) 훈련할 때와 번역할 때: 교사 강요

정의

- 교사 강요(teacher forcing): 훈련할 때, 디코더가 직전 시점에 무엇을 예측했든 상관없이 **정답 단어**를 다음 시점의 입력으로 넣어 주는 방법.

	훈련할 때	번역할 때 (테스트)
디코더의 입력	정답 문장: <sos> je suis étudiant	<sos>, 그다음부터는 자기가 낸 단어
정답(레이블)	한 칸 밀린 문장: je suis étudiant <eos>	없음
계산	모든 시점을 한꺼번에	한 단어씩, <eos>가 나올 때까지 반복

왜 정답을 넣어 줄까? 학습 초기의 디코더는 엉뚱한 단어를 냅니다. 그 엉뚱한 단어를 다음 입력으로 쓰면 뒤의 예측이 줄줄이 틀려, 무엇을 고쳐야 할지 배우기 어렵습니다. 정답을 넣어 주면 시점마다 "올바른 앞 단어가 주어졌을 때 다음 단어 맞히기"를 따로따로 연습할 수 있습니다. 한꺼번에 계산할 수 있어 빠르기도 합니다.

그래서 번역 모델의 학습 데이터는 세 가지입니다. **인코더의 입력**(영어 문장), **디코더의 입력**(<sos> + 프랑스어 문장), **디코더의 정답**(프랑스어 문장 + <eos>).

교사 강요의 부작용. 훈련 때는 늘 올바른 앞 단어를 받았는데, 번역할 때는 자기가 낸(틀릴 수 있는) 단어를 받습니다. 한 번 틀리면 처음 보는 상황이 되어 뒤가 함께 무너질 수 있습니다.

예시 적용

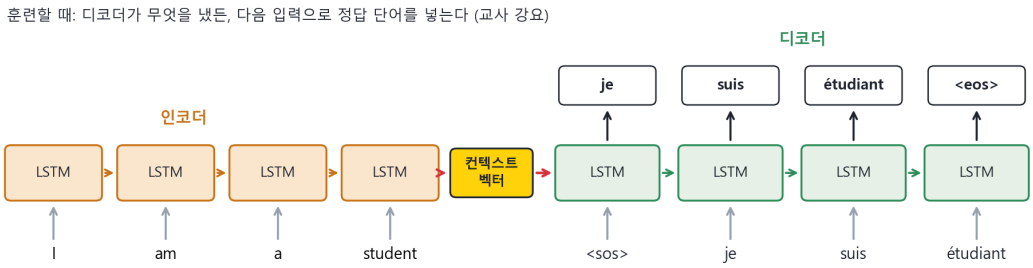


그림 2 훈련할 때는 디코더의 입력으로 정답 문장을 그대로 넣는다. 위 그림과 달리 빨간 점선이 없다

학습되지 않은 작은 모델로 흐름과 모양을 확인합니다(실습 1). 임베딩 차원 8, 은닉 크기 16, 프랑스어 단어 집합 6개입니다.

단계	모양	뜻
인코더 입력	(1, 4)	문장 1개, 단어 4개
컨텍스트 벡터 (hidden, cell)	(1, 1, 16), (1, 1, 16)	(층 수, 문장 수, 은닉 크기). 문장의 길이는 사라졌다
[훈련] 디코더 입력 → 점수	(1, 4) → (1, 4, 6)	자리마다 프랑스어 단어 6개의 점수
[번역] 디코더 입력	(1, 1)	한 번에 단어 하나씩

- 훈련 손실 1.8067은 단어 6개 중 찍을 때의 손실 $\ln 6 = 1.7918$과 비슷합니다. 아직 아무것도 배우지 않았기 때문입니다.
- 번역 흐름에서는 <sos> → étudiant → étudiant → je → je처럼 엉뚱한 단어가 나옵니다. "낸 단어가 다음 입력이 된다"는 흐름만 봅니다.

비유

전화로 듣고, 메모 한 장으로 전하는 통역

한 사람이 영어 문장을 끝까지 듣고 핵심을 메모 한 장에 적습니다. 다른 사람은 그 메모만 받아 프랑스어로 한 단어씩 말합니다. 말하는 사람은 원문을 듣지 못했습니다. 신입일 때는 옆에서 선배가 "지금까지는 이렇게 말했어야 해"라고 올바른 말을 계속 일러 주며 연습시킵니다. 실전에서는 선배가 없으니 자기가 방금 한 말에 이어서 말해야 합니다.

듣는 사람	= = 인코더
메모 한 장	= = 컨텍스트 벡터
말하는 사람	= = 디코더
선배가 일러 주는 올바른 말	= = 교사 강요
자기 말에 이어서 말하기	= = 번역할 때의 디코더

실습 1

인코더와 디코더를 이어 보고, 훈련과 번역의 흐름 비교하기

```python
import torch
import torch.nn as nn

torch.manual_seed(0)
src_vocab = {"<pad>": 0, "i": 1, "am": 2, "a": 3, "student": 4}
tar_vocab = {"<pad>": 0, "<sos>": 1, "<eos>": 2, "je": 3, "suis": 4, "étudiant": 5}
tar_words = {i: w for w, i in tar_vocab.items()}
D, H = 8, 16                                        # 임베딩 차원, 은닉 크기

enc_embedding = nn.Embedding(len(src_vocab), D, padding_idx=0)
enc_lstm = nn.LSTM(D, H, batch_first=True)
dec_embedding = nn.Embedding(len(tar_vocab), D, padding_idx=0)
dec_lstm = nn.LSTM(D, H, batch_first=True)
fc = nn.Linear(H, len(tar_vocab))                   # 프랑스어 단어마다의 점수

# 1) 인코더: 입력 문장을 다 읽고 마지막 은닉 상태와 셀 상태만 넘긴다
src = torch.tensor([[1, 2, 3, 4]])                  # i am a student
_, (hidden, cell) = enc_lstm(enc_embedding(src))
print("인코더 입력:", tuple(src.shape))
print("컨텍스트 벡터 (hidden, cell):", tuple(hidden.shape), tuple(cell.shape))

# 2) 훈련할 때의 디코더 (교사 강요): 정답 문장을 한꺼번에 넣는다
dec_input = torch.tensor([[1, 3, 4, 5]])            # <sos> je suis étudiant
dec_target = torch.tensor([[3, 4, 5, 2]])           # je suis étudiant <eos>
# 디코더의 초기 상태 = 컨텍스트 벡터
out, _ = dec_lstm(dec_embedding(dec_input), (hidden, cell))
logits = fc(out)
print("\n[훈련] 디코더 입력:", tuple(dec_input.shape),
      "→ 점수:", tuple(logits.shape))
flat = logits.view(-1, len(tar_vocab))
loss = nn.CrossEntropyLoss()(flat, dec_target.view(-1))
print(f"       자리마다 다음 단어를 맞히는 분류 4문제의 평균 손실: {loss:.4f}")
guess = torch.log(torch.tensor(6.0))
print(f"       단어 6개 중 찍을 때의 손실 ln 6 = {guess:.4f}")

# 3) 테스트할 때의 디코더: <sos>에서 시작해, 낸 단어를 다음 입력으로 넣는다
print("\n[테스트] 학습 전이라 엉뚱한 단어가 나온다. 흐름만 본다")
token, state = torch.tensor([[1]]), (hidden, cell)
for step in range(1, 6):
    out, state = dec_lstm(dec_embedding(token), state)   # 한 단어씩, 상태를 이어서
    next_id = fc(out)[0, -1].argmax().item()
    print(f"  {step}단계: 입력 '{tar_words[token.item()]}' {tuple(token.shape)}",
```

```
            f"→ 예측 '{tar_words[next_id]}'")
    if next_id == 2:                                # <eos>가 나오면 끝
        break
    token = torch.tensor([[next_id]])
```

```
실행 결과
인코더 입력: (1, 4)
컨텍스트 벡터 (hidden, cell): (1, 1, 16) (1, 1, 16)

[훈련] 디코더 입력: (1, 4) → 점수: (1, 4, 6)
       자리마다 다음 단어를 맞히는 분류 4문제의 평균 손실: 1.8067
       단어 6개 중 찍을 때의 손실 ln 6 = 1.7918

[테스트] 학습 전이라 엉뚱한 단어가 나온다. 흐름만 본다
  1단계: 입력 '<sos>' (1, 1) → 예측 'étudiant'
  2단계: 입력 'étudiant' (1, 1) → 예측 'étudiant'
  3단계: 입력 'étudiant' (1, 1) → 예측 'je'
  4단계: 입력 'je' (1, 1) → 예측 'je'
  5단계: 입력 'je' (1, 1) → 예측 'je'
```

직접 해 보기 디코더를 시작할 때 `(hidden, cell)`을 넘기지 않으면(영벡터에서 시작하면) 디코더는 입력 문장에 대해 무엇을 알 수 있나요?

확인 문제

16-01 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 seq2seq에서 입력 문장과 출력 문장의 길이는 같아야 한다. (O / X) 이름 _____

2 컨텍스트 벡터는 인코더의 마지막 은닉 상태다. (O / X) 이름 _____

3 번역할 때 디코더의 첫 입력은? 이름 _____

① <eos> ② <sos> ③ 입력 문장의 첫 단어

4 교사 강요에서 디코더의 다음 시점 입력으로 넣는 것은? 이름 _____

① 디코더가 방금 예측한 단어 ② 정답 단어 ③ 무작위 단어

5 디코더는 _____ 토큰이 나오면 단어 만들기를 멈춘다. 이름 _____

6 입력 문장을 읽는 RNN을 _____, 출력 문장을 만드는 RNN을 디코더라고 한다. 이름 _____

7 seq2seq의 동작을 메모 한 장으로 전하는 통역에 빗대어 설명하고, 이 방식의 약점을 하나 말해 보세요. 이름 _____

원문: 16-01 시퀀스투시퀀스 — https://wikidocs.net/65154 (교재에는 실습이 없어 구조와 모양을 확인하는 실습을 추가했습니다)

16-02 Seq2Seq를 이용한 번역기 구현하기

핵심 한 줄

영어-프랑스어 문장 쌍을 인코더 입력, 디코더 입력(<sos> 붙임), 디코더 정답(<eos> 붙임)의 세 텐서로 만들고, LSTM 인코더와 디코더를 교사 강요로 학습시킨 뒤 한 단어씩 번역한다.

왜 배우나

16-01의 구조를 실제 데이터로 끝까지 돌려 봅니다. 데이터를 세 갈래로 준비하는 법, 훈련용 코드와 번역용 코드가 왜 따로 필요한지, 그리고 이 단순한 구조가 어디까지 되고 어디서 틀리는지를 확인합니다.

1) 병렬 말뭉치와 데이터 준비

정의

- 병렬 말뭉치(parallel corpus): 같은 뜻의 문장이 두 언어로 짝지어진 데이터. 14장의 태깅 데이터와 달리 두 문장의 **길이가 달라도 됩니다.**
- 단어 집합을 언어마다 따로 만듭니다. 출력 언어의 단어 집합에는 <sos>와 <eos>가 들어갑니다.
- 패딩도 따로 합니다. 인코더 입력은 영어 문장의 최대 길이로, 디코더 입력과 정답은 프랑스어 문장의 최대 길이로 맞춥니다.

$$(\text{en},\ \text{fr}) \rightarrow \begin{cases} \text{encoder input}: & \text{en} \\ \text{decoder input}: & \texttt{<sos>} + \text{fr} \\ \text{decoder target}: & \text{fr} + \texttt{<eos>} \end{cases}$$

전처리. 프랑스어의 악센트를 없애고(é → e), 소문자로 바꾸고, 구두점을 단어에서 떼어 냅니다("student." → "student ."). 단어 집합을 줄이고 구두점도 하나의 토큰으로 다루기 위해서입니다.

예시 적용

영어-프랑스어 문장 쌍 약 19만 개 가운데 앞 33,000개를 씁니다(실습 2, 교재와 같음). 짧은 문장부터 정렬되어 있어 모두 짧은 문장입니다.

항목	값
문장 쌍	33,000 (학습 26,700 / 검증 3,000 / 테스트 3,300)
영어 단어 집합	4,674
프랑스어 단어 집합	8,087
인코더 입력	(33000, 8)
디코더 입력, 디코더 정답	(33000, 16), (33000, 16)

"sing a song with me ."의 짝은 이렇게 준비됩니다. 디코더 입력과 정답은 한 칸 밀려 있습니다.

인코더 입력	sing a song with me .
디코더 입력	<sos> chante une chanson avec moi .
디코더 정답	chante une chanson avec moi . <eos>

프랑스어 단어 집합이 영어의 1.7배입니다. 프랑스어는 주어와 성별에 따라 단어의 모양이 더 다양하게 바뀌기 때문입니다. 교재의 숫자(4,517 / 7,908)와 조금 다른 것은 내려받은 파일의 판이 달라서입니다.

2) 모델과 학습

정의

- **인코더**: `Embedding → LSTM` . 전체 출력은 버리고 `(hidden, cell)` 만 돌려줍니다.
- **디코더**: `Embedding → LSTM → Linear` . LSTM의 초기 상태로 인코더의 `(hidden, cell)` 을 받습니다. 선형층은 프랑스어 단어 집합 크기만큼의 점수를 냅니다.
- **손실**: 자리마다의 예측과 디코더 정답 사이의 크로스 엔트로피. 패딩 자리는 뺍니다 (`ignore_index=0` , 14-01과 같음).
- **정확도**: 패딩이 아닌 자리 가운데 다음 단어를 맞힌 비율. 교사 강요 상태에서 잰 값이라, 실제 번역의 품질보다 좋게 나옵니다.

번역 코드가 따로 필요한 이유. 훈련할 때의 `forward` 는 디코더 입력 전체를 한 번에 받습니다. 번역할 때는 정답이 없으므로, 인코더를 한 번 돌린 뒤 디코더를 **한 단어씩** 돌리며 은닉 상태와 셀 상태를 다음 단계로 넘겨 줘야 합니다. 그래서 `translate` 함수를 따로 만듭니다.

예시 적용

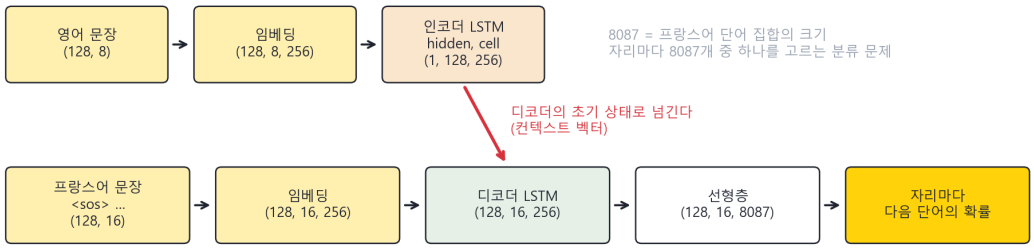


그림 3 배치 128개가 모델을 지날 때의 모양. 인코더에서 디코더로 넘어가는 것은 (1, 128, 256) 크기의 상태뿐이다

임베딩 차원 256, 은닉 크기 256(교재와 같음)으로 10 에포크 학습했습니다(실습 3, 교재는 30 에포크).

에포크	학습 손실	검증 손실	검증 정확도 (교사 강요)
1	4.2205	3.2800	0.4869
3	2.3607	2.3666	0.5998
5	1.7231	2.0293	0.6447
10	0.8070	1.6576	0.7007

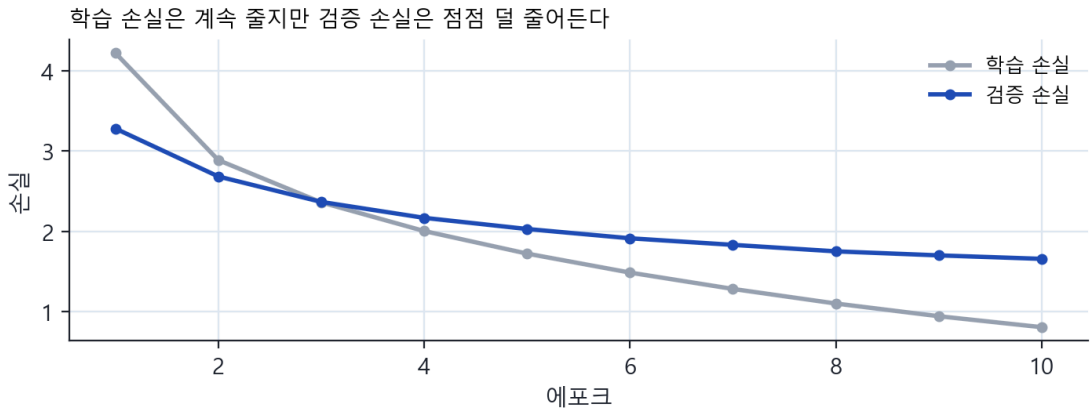


그림 4 10 에포크 동안의 손실. 학습 손실과 검증 손실의 차이가 점점 벌어진다

검증 정확도 0.70은 교재의 10 에포크 결과(0.7120)와 비슷합니다. 교재는 30 에포크까지 학습해 0.7180에서 멈췄습니다. 더 학습해도 크게 오르지 않는다는 뜻입니다.

실제로 번역시켜 보기(한 단어씩 만드는 `translate`). 테스트 데이터의, 모델이 처음 보는 문장들입니다.

입력	정답	모델의 번역	평가
they all did it .	ils le firent tous .	ils l ont fait tous .	뜻은 맞음. 과거를 나타내는 다른 표현을 씀
we all hate tom .	nous detestons tous tom .	nous allons tous tom .	틀림. hate 대신 "간다"(allons)를 넣음
this gold is mine .	cet or est mien .	ce chien a est le mien .	틀림. gold가 "개"(chien)로 바뀜
the door creaked .	la porte grinca .	la porte s ouvrit de la porte .	틀림. "문이 문에서 열렸다"처럼 같은 말을 반복
get out of the truck .	sortez du camion !	sortez de la porte !	틀림. truck 대신 "문"(porte)을 넣음

- **정확도 0.70인데 번역은 대부분 틀립니다.** 테스트 문장 500개 가운데 정답과 완전히 같은 번역은 34개(6.8%)입니다. 0.70은 "올바른 앞 단어를 받았을 때 다음 단어 하나를 맞힐 확률"이고, 문장 전체가 맞으려면 모든 단어를 연달아 맞혀야 합니다.
- **문장의 틀은 배웠습니다.** ils … tous(그들은 모두), sortez de …(…에서 나가라), la porte(문)처럼 문장의 뼈대와 자주 나오는 단어는 맞습니다.
- **드문 단어를 흔한 단어로 바꿉니다.** truck, gold, creaked처럼 학습 데이터에 드물게 나온 단어가 porte(문), chien(개) 같은 흔한 단어로 바뀝니다. 컨텍스트 벡터 하나에 담긴 정보에서 드문 단어의 흔적이 가장 먼저 사라집니다.
- **학습 데이터의 문장도 다 맞히지는 못합니다.** "you hurt me ."의 정답은 "vous me faites mal ."인데 "tu me fais mal ."로 번역했습니다. 프랑스어의 you는 vous(격식)와 tu(친근) 둘 다 되므로 이 번역도 맞습니다. 정답과 글자가 다르다고 해서 틀린 번역은 아닙니다. 이 문제를 16-03에서 다룹니다.

데이터가 33,000쌍뿐이고 문장도 짧습니다. 교재도 "실제 번역기는 어텐션을 써야 하고 최소 수백만 쌍의 데이터가 필요하다"고 밝힙니다.

비유

받아쓰기 연습과 실제 통역

연습할 때는 모범 번역문을 옆에 두고 "여기까지 맞게 썼다면 다음 단어는?"을 단어마다 채점합니다. 점수가 잘 나옵니다. 실제 통역에서는 모범 답안이 없고, 한 단어를 잘못 말하면 그 뒤의 문장이 다른 길로 갑니다. 연습 점수와 실전 실력이 다른 이유입니다.

모범 번역문을 보며 채점 = = 교사 강요로 잰 정확도

실제 통역 = = translate 함수 (한 단어씩 생성)

한 단어를 잘못 말한 뒤 = = 틀린 단어가 다음 입력이 됨

실습 2

영어-프랑스어 문장 쌍을 세 개의 텐서로 준비하기

```
# 영어-프랑스어 문장 쌍을 내려받아 정수 텐서로 준비한다 (3MB)
import io
import re
import unicodedata
import urllib.request
import zipfile
from collections import Counter

import torch

# 교재의 주소(manythings.org)는 프로그램으로 받으면 막힐 때가 있어, 같은 자료의
# 다른 보관 주소를 쓴다
url = "https://storage.googleapis.com/download.tensorflow.org/data/fra-eng.zip"
zipfile.ZipFile(io.BytesIO(urllib.request.urlopen(url).read())).extract(
    "fra.txt")
num_samples = 33000                                    # 교재와 같이 앞 33,000쌍만

def preprocess(sentence):
    # 악센트 제거(é → e), 소문자로, 구두점 앞에 공백, 그 밖의 기호 제거
    s = unicodedata.normalize("NFD", sentence.lower())
    s = "".join(c for c in s if unicodedata.category(c) != "Mn")
    s = re.sub(r"([?.!,])", r" \1", s)
    s = re.sub(r"[^a-z!.?]+", " ", s)
    return s.split()

pairs = []
for line in open("fra.txt", encoding="utf8").read().split("\n")[:num_samples]:
    en, fr = line.split("\t")[:2]
    pairs.append((preprocess(en), preprocess(fr)))
print("문장 쌍:", len(pairs))
for i in [0, 100, 30000]:
    print(f"  {' '.join(pairs[i][0]):28s} ↔ {' '.join(pairs[i][1])}")

# 디코더의 입력은 앞에 <sos>를, 정답은 뒤에 <eos>를 붙인다
enc_in = [en for en, _ in pairs]
dec_in = [["<sos>"] + fr for _, fr in pairs]
dec_out = [fr + ["<eos>"] for _, fr in pairs]

def build_vocab(sentences):
    counts = Counter(w for s in sentences for w in s)
    words = sorted(counts, key=counts.get, reverse=True)
    return {"<PAD>": 0, "<UNK>": 1, **{w: i + 2 for i, w in enumerate(words)}}

src_vocab = build_vocab(enc_in)
tar_vocab = build_vocab(dec_in + dec_out)              # <sos>, <eos>도 들어간다
print("영어 단어 집합:", len(src_vocab), " 프랑스어 단어 집합:", len(tar_vocab))

def encode(sentences, vocab):
    length = max(len(s) for s in sentences)
    out = torch.zeros(len(sentences), length, dtype=torch.long)
    for i, s in enumerate(sentences):
        out[i, :len(s)] = torch.tensor([vocab[w] for w in s])
    return out

X, Y_in, Y_out = encode(enc_in, src_vocab), encode(dec_in, tar_vocab), encode(
    dec_out, tar_vocab)
print("인코더 입력:", tuple(X.shape), " 디코더 입력:", tuple(Y_in.shape),
      " 정답:", tuple(Y_out.shape))
```

```
tar_words = {i: w for w, i in tar_vocab.items()}
print("디코더 입력 예:", [tar_words[i] for i in Y_in[30000].tolist() if i])
print("그 정답       :", [tar_words[i] for i in Y_out[30000].tolist() if i])

# 섞은 뒤 10%를 테스트용으로 뗀다
torch.manual_seed(0)
order = torch.randperm(num_samples)
n_test = num_samples // 10
test, train = order[:n_test], order[n_test:]
data = {"src_vocab": src_vocab, "tar_vocab": tar_vocab}
for name, idx in [("train", train), ("test", test)]:
    data[name] = (X[idx], Y_in[idx], Y_out[idx])
torch.save(data, "seq2seq_data.pt")
print("학습:", len(train), " 테스트:", len(test))
```

```
실행 결과
문장 쌍: 33000
  go .                        ↔ va !
  come in .                   ↔ entrez !
  sing a song with me .       ↔ chante une chanson avec moi .
영어 단어 집합: 4674  프랑스어 단어 집합: 8087
인코더 입력: (33000, 8)  디코더 입력: (33000, 16)  정답: (33000, 16)
디코더 입력 예: ['<sos>', 'chante', 'une', 'chanson', 'avec', 'moi', '.']
그 정답       : ['chante', 'une', 'chanson', 'avec', 'moi', '.', '<eos>']
학습: 29700  테스트: 3300
```

직접 해 보기 `preprocess("Où est la bibliothèque ?")` 를 실행해 보세요. 악센트와 물음표가 어떻게 처리되나요?

실습 3 seq2seq 번역기 학습하고 번역하기

```
# 앞 실습(데이터 준비)을 먼저 실행해 둔다 (CPU로 약 15분)
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

torch.manual_seed(0)
data = torch.load("seq2seq_data.pt")
src_vocab, tar_vocab = data["src_vocab"], data["tar_vocab"]
src_words = {i: w for w, i in src_vocab.items()}
tar_words = {i: w for w, i in tar_vocab.items()}
SOS, EOS = tar_vocab["<sos>"], tar_vocab["<eos>"]

class Encoder(nn.Module):
    def __init__(self, vocab_size, embedding_dim=256, hidden_units=256):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
        self.lstm = nn.LSTM(embedding_dim, hidden_units, batch_first=True)

    def forward(self, x):
        _, (hidden, cell) = self.lstm(self.embedding(x))
        return hidden, cell                    # 컨텍스트 벡터: 각각 (1, 배치, 256)

class Decoder(nn.Module):
    def __init__(self, vocab_size, embedding_dim=256, hidden_units=256):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
        self.lstm = nn.LSTM(embedding_dim, hidden_units, batch_first=True)
        self.fc = nn.Linear(hidden_units, vocab_size)

    def forward(self, x, hidden, cell):
        # 인코더가 넘긴 상태에서 시작한다
        output, (hidden, cell) = self.lstm(self.embedding(x), (hidden, cell))
        # 점수의 모양: (배치, 길이, 프랑스어 단어 수)
        return self.fc(output), hidden, cell

class Seq2Seq(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = Encoder(len(src_vocab))
        self.decoder = Decoder(len(tar_vocab))
```

```
    def forward(self, src, dec_input):          # 훈련용: 교사 강요
        hidden, cell = self.encoder(src)
        logits, _, _ = self.decoder(dec_input, hidden, cell)
        return logits

model = Seq2Seq()
criterion = nn.CrossEntropyLoss(ignore_index=0)
optimizer = torch.optim.Adam(model.parameters())
X, Y_in, Y_out = data["train"]
n_valid = 3000                                  # 학습 데이터의 끝 3,000쌍은 검증용
train_set = TensorDataset(X[:-n_valid], Y_in[:-n_valid], Y_out[:-n_valid])
valid = (X[-n_valid:], Y_in[-n_valid:], Y_out[-n_valid:])
print("모델 출력:", tuple(model(X[:2], Y_in[:2]).shape))

def evaluate(src, dec_input, target):
    model.eval()
    with torch.no_grad():
        logits = model(src, dec_input)
    loss = criterion(logits.view(-1, logits.size(-1)), target.view(-1)).item()
    mask = target != 0
    acc = (logits.argmax(-1) == target)[mask].float().mean().item()
    return loss, acc

history, best = [], float("inf")
for epoch in range(1, 11):
    model.train()
    total = 0.0
    loader = DataLoader(train_set, batch_size=128, shuffle=True)
    for src, dec_input, target in loader:
        logits = model(src, dec_input)
        loss = criterion(logits.view(-1, logits.size(-1)), target.view(-1))
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total += loss.item() * len(src)
    val_loss, val_acc = evaluate(*valid)
    history.append((total / len(train_set), val_loss, val_acc))
    saved = ""
    if val_loss < best:
        best, saved = val_loss, "  ← 저장"
        torch.save(model.state_dict(), "seq2seq.pth")
    print(f"에포크 {epoch:2d}: 학습 손실 {history[-1][0]:.4f} |",
          f"검증 손실 {val_loss:.4f}, 정확도 {val_acc:.4f}{saved}")
model.load_state_dict(torch.load("seq2seq.pth"))

def translate(src_ids, max_len=20):
    """<sos>에서 시작해 한 단어씩 만들고, 만든 단어를 다음 입력으로 넣는다"""
    model.eval()
    with torch.no_grad():
        hidden, cell = model.encoder(src_ids.unsqueeze(0))
        token, words = torch.tensor([[SOS]]), []
        for _ in range(max_len):
            logits, hidden, cell = model.decoder(token, hidden, cell)
            next_id = logits[0, -1].argmax().item()
            if next_id == EOS:
                break
            words.append(tar_words[next_id])
            token = torch.tensor([[next_id]])
    return words

def show(src_ids, target_ids):
    source = " ".join(src_words[i] for i in src_ids.tolist() if i)
    answer = [tar_words[i] for i in target_ids.tolist() if i and i != EOS]
    print(f"  입력: {source:26s} 정답: {' '.join(answer):30s}",
          f"번역: {' '.join(translate(src_ids))}")
    return answer

print("\n학습 데이터의 문장")
for i in [3, 50, 100, 300, 1001]:
    show(X[i], Y_out[i])
print("테스트 데이터의 문장 (처음 보는 문장)")
Xt, _, Yt = data["test"]
for i in [0, 1, 2, 3, 4, 5]:
    show(Xt[i], Yt[i])

# BLEU 계산(다음 소챕터)을 위해 테스트 문장 500개의 정답과 번역을 저장한다
refs = [[tar_words[i] for i in y.tolist() if i and i != EOS] for y in Yt[:500]]
```

```
hyps = [translate(x) for x in Xt[:500]]
exact = sum(r == h for r, h in zip(refs, hyps))
print(f"테스트 500문장 중 정답과 완전히 같은 번역: {exact}개")
torch.save({"history": history, "refs": refs, "hyps": hyps}, "seq2seq_result.pt")
```

```
실행 결과
모델 출력: (2, 16, 8087)
에포크  1: 학습 손실 4.2205 | 검증 손실 3.2800, 정확도 0.4869  ← 저장
에포크  2: 학습 손실 2.8870 | 검증 손실 2.6833, 정확도 0.5609  ← 저장
에포크  3: 학습 손실 2.3607 | 검증 손실 2.3666, 정확도 0.5998  ← 저장
에포크  4: 학습 손실 2.0041 | 검증 손실 2.1683, 정확도 0.6264  ← 저장
에포크  5: 학습 손실 1.7231 | 검증 손실 2.0293, 정확도 0.6447  ← 저장
에포크  6: 학습 손실 1.4866 | 검증 손실 1.9138, 정확도 0.6613  ← 저장
에포크  7: 학습 손실 1.2830 | 검증 손실 1.8319, 정확도 0.6700  ← 저장
에포크  8: 학습 손실 1.1015 | 검증 손실 1.7511, 정확도 0.6853  ← 저장
에포크  9: 학습 손실 0.9435 | 검증 손실 1.7010, 정확도 0.6918  ← 저장
에포크 10: 학습 손실 0.8070 | 검증 손실 1.6576, 정확도 0.7007  ← 저장

학습 데이터의 문장
  입력: stand aside .              정답: ecartez vous .                번역: restez a l
ecart .
  입력: this is ours .             정답: c est a nous .                번역: c est la
mienne .
  입력: we re the best .           정답: nous sommes les meilleurs .   번역: nous sommes
les meilleurs .
  입력: compare the facts .        정답: compare les faits !           번역: allume les
portes !
  입력: you hurt me .              정답: vous me faites mal .          번역: tu me fais
mal .
테스트 데이터의 문장 (처음 보는 문장)
  입력: they all did it .          정답: ils le firent tous .          번역: ils l ont
fait tous .
  입력: we all hate tom .          정답: nous detestons tous tom .     번역: nous allons
tous tom .
  입력: that seemed to help .      정답: ca a paru aider .             번역: ca aide a l
aide .
  입력: this gold is mine .        정답: cet or est mien .             번역: ce chien a
est le mien .
  입력: the door creaked .         정답: la porte grinca .             번역: la porte s
ouvrit de la porte .
  입력: get out of the truck .     정답: sortez du camion !            번역: sortez de la
porte !
테스트 500문장 중 정답과 완전히 같은 번역: 34개
```

직접 해 보기 직접 번역시켜 보세요. 영어 문장을 `preprocess` 와 같은 방식으로 자르고 `src_vocab` 으로 번호를 붙여 `translate(torch.tensor([...]))` 에 넣으면 됩니다. 단어 집합에 없는 단어는 어떻게 해야 할까요?

확인 문제

16-02 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 병렬 말뭉치에서 두 언어의 문장은 길이가 같아야 한다. (O / X) 이름 _____

2 인코더는 LSTM의 모든 시점의 출력을 디코더에 넘긴다. (O / X) 이름 _____

3 디코더의 정답(레이블)으로 맞는 것은? 이름 _____

① <sos> je suis étudiant ② je suis étudiant <eos> ③ I am a student

4 디코더의 선형층이 내는 점수의 개수는? 이름 _____

① 영어 단어 집합의 크기 ② 프랑스어 단어 집합의 크기 ③ 은닉 크기

5 디코더의 입력은 번역문의 맨 앞에 _____ 토큰을 붙인 것이다. 이름 _____

6 실습의 프랑스어 단어 집합의 크기는 _____이다. 이름 _____

7 훈련 때 잰 정확도가 실제 번역의 품질보다 좋게 나오는 이유를 받아쓰기 연습과 실제 통역에 빗대어 설명해 보세요. 이름 _____

원문: 16-02 Seq2Seq를 이용한 번역기 구현하기 — https://wikidocs.net/216494 (교재의 데이터 주소가 프로그램으로 받을 때 막혀 같은 자료의 다른 보관 주소를 썼고, 30 에포크를 10 에포크로 줄였습니다. 검증 데이터는 학습 데이터에서 3,000쌍을 떼어 썼습니다)

16-03 BLEU Score

핵심 한 줄

BLEU는 기계 번역이 사람의 번역과 n-gram을 얼마나 공유하는지를 세어 0~1의 점수로 나타내며, 같은 단어의 반복과 너무 짧은 번역이 점수를 얻지 못하도록 두 가지 보정을 한다.

왜 배우나

번역에는 정답이 하나가 아닙니다. "tu es chanceux"와 "tu as de la chance"는 둘 다 "너는 운이 좋다"입니다. 정답과 글자가 똑같은지만 보면 좋은 번역을 틀렸다고 하게 됩니다. 사람이 일일이 읽고 채점하면 정확하지만 느리고 비쌉니다. 그래서 자동으로, 빠르게, 그럴듯하게 점수를 매기는 방법이 필요하고, 가장 널리 쓰이는 것이 BLEU입니다.

1) 정밀도에서 시작해 허점을 메워 가기

정의

- BLEU(Bilingual Evaluation Understudy): 기계 번역(Candidate, Ca)을 사람이 번역한 정답(Reference, Ref)들과 비교해 점수를 내는 방법. **높을수록 좋습니다**(10-03의 PPL과 반대). 언어와 상관없이 쓸 수 있고 계산이 빠릅니다.

(1) 유니그램 정밀도. 가장 단순한 방법은 "번역의 단어 중 정답 어딘가에 나오는 단어의 비율"을 재는 것입니다(06-01의 정밀도).

$$\text{unigram precision} = \frac{\text{number of Ca words that appear in any Ref}}{\text{number of words in Ca}}$$

허점: "the the the the the the the"는 the가 정답에 있으므로 7/7 = 1점 만점입니다.

(2) 보정된 정밀도. 한 단어를 맞힌 것으로 쳐 주는 횟수를, 그 단어가 **정답 하나에 나온 최대 횟수**까지로 자릅니다.

$$\text{Count}_{clip} = \min(\text{Count},\ \text{Max_Ref_Count}), \qquad p_n = \frac{\sum \text{Count}_{clip}(\text{n-gram})}{\sum \text{Count}(\text{n-gram})}$$

Count는 번역에 그 n-gram이 나온 횟수, Max_Ref_Count는 정답 문장 하나에 가장 많이 나온 횟수입니다. p_n은 n-gram 기준의 보정된 정밀도입니다.

허점: 단어만 세면 순서를 보지 않습니다. 단어를 뒤죽박죽 섞어도 점수가 같습니다.

(3) n-gram으로 넓히기. 단어 하나(1-gram)뿐 아니라 이웃한 2개, 3개, 4개 묶음(10-02의 n-gram)으로도 같은 정밀도를 구해 함께 봅니다. 묶음이 길수록 순서까지 맞아야 합니다. 네 값은 **기하 평균**으로 합칩니다.

허점: 정밀도는 "낸 것 중에 맞힌 비율"이라, 확실한 단어 몇 개만 짧게 내면 높게 나옵니다.

(4) 길이 벌점(brevity penalty, BP). 번역이 정답보다 짧으면 점수를 깎습니다.

$$BP = \begin{cases} 1 & (c > r) \\ e^{\,1-r/c} & (c \le r) \end{cases} \qquad BLEU = BP \times \exp\left(\sum_{n=1}^{4} \frac{1}{4} \log p_n\right)$$

c는 번역의 길이, r은 정답의 길이(정답이 여럿이면 번역과 길이가 가장 가까운 것)입니다. $\exp(\sum \frac{1}{4}\log p_n)$은 p_1, p_2, p_3, p_4의 기하 평균, 곧 넷을 곱해 네제곱근을 씌운 값입니다.

- **왜 산술 평균이 아니라 기하 평균일까?** 곱으로 합치면 하나라도 0이면 전체가 0이 됩니다. 단어는 맞는데 순서가 전혀 안 맞는 번역이 높은 점수를 받지 못하게 합니다.
- **그 부작용:** 짧은 문장은 4-gram이 하나도 겹치지 않아 0점이 되기 쉽습니다. 그래서 BLEU는 문장 하나가 아니라 **문서나 테스트 세트 전체**의 n-gram을 모두 합쳐 계산하는 것이 원칙입니다.

예시 적용

교재(BLEU 논문)의 예입니다. 정답 번역 3개와 기계 번역 2개가 있습니다(실습 4).

- **Ca1**: It is a guide to action which ensures that the military always obeys the commands of the party
- **Ca2**: It is to insure the troops forever hearing the activity guidebook that party direct
- **Ref1**: It is a guide to action that ensures that the military will forever heed Party commands

(나머지 정답 2개는 실습 코드에)

	1-gram	2-gram	3-gram	4-gram	길이 벌점	BLEU
Ca1	17/18	10/17	7/16	4/15	1.0	0.5046
Ca2	8/14	1/13	0/12	0/11	-	0

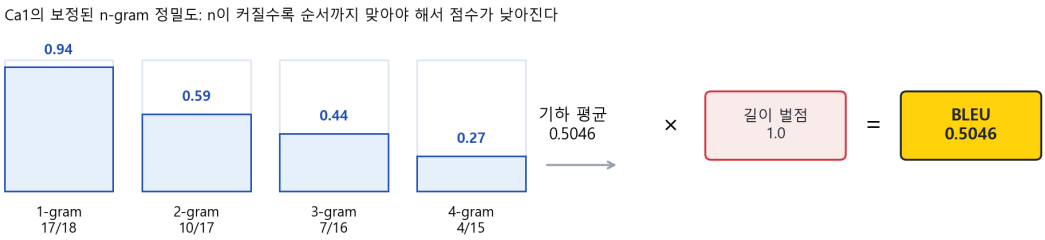


그림 5 Ca1의 BLEU. 네 정밀도의 기하 평균에 길이 벌점을 곱한다

- **the × 7**: 단순 정밀도는 7/7이지만 보정하면 2/7입니다. the가 정답 하나에 최대 2번 나오기 때문입니다.
- **Ca1**: 단어는 18개 중 17개가 정답에 있지만(obeys만 없음), 4개 묶음은 15개 중 4개만 겹칩니다. n이 커질수록 낮아집니다.
- **Ca2**: 단어는 14개 중 8개가 맞지만 3개 묶음이 하나도 겹치지 않아 BLEU는 0입니다. 단어는 비슷해도 어순이 엉망이라는 뜻입니다.
- **"it is"**: 1-gram 정밀도는 1/2로 나쁘지 않지만 길이 벌점이 0.0009입니다. 정답(16단어)에 비해 터무니없이 짧기 때문입니다.
- 직접 계산한 값이 NLTK의 `sentence_bleu`와 같습니다.

2) 우리 번역기의 BLEU와 한계

정의

- 말뭉치 BLEU(corpus BLEU): 테스트 문장 전체의 n-gram 수를 합쳐 한 번에 계산한 BLEU. 문장마다 재서 평균한 것과 다릅니다.
- 스무딩(smoothing): 문장 하나의 BLEU를 볼 때, 겹치는 n-gram이 0개여도 점수가 0이 되지 않도록 작은 값을 더해 주는 보정.

BLEU의 한계.

- 뜻이 같아도 단어가 다르면 점수를 주지 않습니다. 비슷한 말(동의어)을 모릅니다.
- 정답이 하나뿐이면 다른 올바른 번역이 낮은 점수를 받습니다. 그래서 정답을 여러 개 두는 것이 좋습니다.
- 뜻을 정반대로 만드는 단어 하나(not)가 빠져도 점수는 조금만 떨어집니다.
- 그래서 BLEU는 "두 모델 중 어느 쪽이 나은가"를 비교하는 데 쓰고, 점수 하나만으로 번역이 좋다 나쁘다를 말하지는 않습니다.

예시 적용

16-02에서 만든 번역기로 테스트 문장 500개를 번역해 말뭉치 BLEU를 쟀습니다(실습 5). 정답은 문장마다 하나뿐입니다.

어디까지 보나	1-gram	1~2-gram	1~3-gram	1~4-gram (보통의 BLEU)
BLEU	0.5880	0.4247	0.3211	0.2471

- **BLEU 0.2471**(100점 만점으로 24.7)입니다. 단어만 보면 0.59로 절반 넘게 겹치지만, 긴 묶음까지 보면 낮아집니다. 단어는 그럴듯한데 구절 단위로는 정답과 다르다는 뜻입니다.
- 이 숫자 하나로는 좋은지 나쁜지 말하기 어렵습니다. 17장에서 어텐션을 더한 번역기의 BLEU와 비교합니다.

문장 하나씩 보면 BLEU의 한계가 보입니다(스무딩을 쓴 문장 BLEU).

정답	번역	문장 BLEU	실제로는
ils le firent tous .	ils l ont fait tous .	0.096	뜻이 맞는 번역
nous detestons tous tom .	nous allons tous tom .	0.286	뜻이 틀린 번역

- **뜻이 맞는 번역이 더 낮은 점수를 받았습니다.** "ils l'ont fait tous"는 "그들은 모두 그것을 했다"로 맞는 번역이지만, 정답과 다른 과거 표현을 써서 겹치는 묶음이 적습니다(0.096).
- "nous allons tous tom"은 hate가 빠져 뜻이 틀렸는데, 다섯 단어 중 네 개가 정답과 같아 점수가 더 높습니다(0.286).
- 정답과 완전히 같은 번역은 500개 중 34개뿐이지만, 위의 예처럼 글자는 달라도 맞는 번역이 섞여 있습니다. BLEU도 완전 일치율도 번역의 품질을 그대로 보여 주지는 않습니다.

비유

모범 답안과 겹치는 구절에 밑줄 긋기

서술형 답안을 채점할 때, 모범 답안과 겹치는 낱말과 구절에 밑줄을 긋고 그 비율로 점수를 줍니다. 같은 낱말만 반복해 쓴 답안은 한 번만 쳐 주고, 긴 구절이 겹칠수록 더 쳐 주고, 한 줄만 쓴 답안은 감점합니다. 이 방법은 빠르고 공정해 보이지만, 모범 답안과 다른 말로 쓴 훌륭한 답안에는 낮은 점수를 줍니다.

모범 답안	= = Reference
겹치는 낱말·구절의 비율	= = n-gram 정밀도
반복은 한 번만 쳐 주기	= = Count clip
한 줄 답안 감점	= = 길이 벌점
다른 말로 쓴 좋은 답안	= = BLEU의 한계

실습 4

BLEU를 식 그대로 구현하고 NLTK와 맞춰 보기

```
import math
import warnings
from collections import Counter

from nltk.translate.bleu_score import sentence_bleu

warnings.filterwarnings("ignore")        # NLTK의 "0점" 경고를 끈다

def ngrams(tokens, n):
    return [tuple(tokens[i:i + n]) for i in range(len(tokens) - n + 1)]

def modified_precision(candidate, references, n):
    """보정된 n-gram 정밀도: 정답에 나온 횟수까지만 맞힌 것으로 쳐 준다"""
    counts = Counter(ngrams(candidate, n))
    clipped = 0
    for gram, count in counts.items():
        max_ref = max(Counter(ngrams(ref, n))[gram] for ref in references)
        clipped += min(count, max_ref)                # Count_clip
    return clipped, max(sum(counts.values()), 1)

def brevity_penalty(candidate, references):
    """번역이 정답보다 짧으면 깎는다"""
    c = len(candidate)
    r = min((len(ref) for ref in references), key=lambda x: (abs(x - c), x))
    return 1.0 if c > r else math.exp(1 - r / c)

def bleu(candidate, references, max_n=4):
    logs = 0.0
    for n in range(1, max_n + 1):
        match, total = modified_precision(candidate, references, n)
        if match == 0:
            return 0.0
        logs += math.log(match / total) / max_n       # 기하 평균
    return brevity_penalty(candidate, references) * math.exp(logs)

refs = [s.split() for s in [
    "It is a guide to action that ensures that the military will forever heed "
    "Party commands",
    "It is the guiding principle which guarantees the military forces always being "
    "under the command of the Party",
    "It is the practical guide for the army always to heed the directions of the "
    "party"]]
ca1 = ("It is a guide to action which ensures that the military always obeys the "
       "commands of the party").split()
ca2 = ("It is to insure the troops forever hearing the activity guidebook that "
       "party direct").split()

# 1) 단어만 세면 엉터리 번역이 만점을 받는다 → 횟수를 정답 기준으로 자른다
the7 = "the the the the the the the".split()
cat_refs = ["the cat is on the mat".split(), "there is a cat on the mat".split()]
plain = sum(w in cat_refs[0] or w in cat_refs[1] for w in the7)
print("the × 7:  단순 정밀도", f"{plain}/7", " 보정된 정밀도",
```

```
      "%d/%d" % modified_precision(the7, cat_refs, 1))

# 2) n을 키우면 순서까지 본다
for name, ca in [("Ca1", ca1), ("Ca2", ca2)]:
    parts = ["%d/%d" % modified_precision(ca, refs, n) for n in range(1, 5)]
    print(f"{name}: 1~4-gram 보정 정밀도 {parts}")

# 3) 짧은 번역은 정밀도가 높게 나온다 → 길이 벌점
short = "it is".split()
p = ["%d/%d" % modified_precision(short, refs, n) for n in (1, 2)]
print(f"'it is': 정밀도 {p}, 길이 벌점 {brevity_penalty(short, refs):.4f}")

# 4) BLEU = 길이 벌점 × (1~4-gram 정밀도의 기하 평균). NLTK와 비교
for name, ca in [("Ca1", ca1), ("Ca2", ca2)]:
    mine, theirs = bleu(ca, refs), sentence_bleu(refs, ca)
    print(f"{name}: 직접 계산 {mine:.4f} / NLTK {theirs:.4f}",
          "같다" if abs(mine - theirs) < 1e-6 else "다르다")
```

```
실행 결과
the × 7:  단순 정밀도 7/7  보정된 정밀도 2/7
Ca1: 1~4-gram 보정 정밀도 ['17/18', '10/17', '7/16', '4/15']
Ca2: 1~4-gram 보정 정밀도 ['8/14', '1/13', '0/12', '0/11']
'it is': 정밀도 ['1/2', '0/1'], 길이 벌점 0.0009
Ca1: 직접 계산 0.5046 / NLTK 0.5046 같다
Ca2: 직접 계산 0.0000 / NLTK 0.0000 같다
```

직접 해 보기 `bleu(ca1[:9], refs)` 처럼 번역을 앞의 절반만 잘라 넣어 보세요. 정밀도와 길이 벌점이 각각 어떻게 달라지나요?

실습 5 우리 번역기의 BLEU 재기

```
# 실습 3(번역기 학습)이 저장한 테스트 문장 500개의 번역으로 BLEU를 잰다
import torch
from nltk.translate.bleu_score import SmoothingFunction, corpus_bleu, sentence_bleu

result = torch.load("seq2seq_result.pt")
refs, hyps = result["refs"], result["hyps"]

# 1) 말뭉치 전체의 BLEU: 문장마다 재서 평균하지 않고, n-gram 수를 모두 합쳐 계산
score = corpus_bleu([[r] for r in refs], hyps)
print(f"테스트 500문장의 BLEU: {score:.4f}  (100점 만점으로 {score * 100:.1f})")
for n in range(1, 5):
    weights = tuple(1 / n for _ in range(n))
    part = corpus_bleu([[r] for r in refs], hyps, weights=weights)
    print(f"  1~{n}-gram까지만 본 BLEU: {part:.4f}")

# 2) 문장 하나씩 보기: 짧은 문장은 4-gram이 없어 0점이 되기 쉽다
smooth = SmoothingFunction().method1                # 0이 되지 않게 보정
print("\n정답 / 번역 / 문장 BLEU")
for i in [0, 1, 2, 3, 4, 5]:
    s = sentence_bleu([refs[i]], hyps[i], smoothing_function=smooth)
    print(f"  {' '.join(refs[i]):30s} / {' '.join(hyps[i]):30s} / {s:.3f}")

# 3) BLEU의 한계: 뜻이 같아도 단어가 다르면 점수가 낮다
exact = sum(r == h for r, h in zip(refs, hyps))
print(f"\n정답과 완전히 같은 번역: {exact}개 / 500개")
```

```
실행 결과
테스트 500문장의 BLEU: 0.2471  (100점 만점으로 24.7)
  1~1-gram까지만 본 BLEU: 0.5880
  1~2-gram까지만 본 BLEU: 0.4247
  1~3-gram까지만 본 BLEU: 0.3211
  1~4-gram까지만 본 BLEU: 0.2471

정답 / 번역 / 문장 BLEU
  ils le firent tous .           / ils l ont fait tous .          / 0.096
  nous detestons tous tom .      / nous allons tous tom .         / 0.286
  ca a paru aider .              / ca aide a l aide .             / 0.054
```

```
cet or est mien .              / ce chien a est le mien .        / 0.077
la porte grinca .              / la porte s ouvrit de la porte . / 0.065
sortez du camion !             / sortez de la porte !            / 0.064

정답과 완전히 같은 번역: 34개 / 500개
```

직접 해 보기 정답과 번역이 다른 문장을 몇 개 골라 직접 읽어 보세요(프랑스어 번역기를 써도 좋습니다). BLEU는 낮지만 뜻은 맞는 번역이 있나요?

확인 문제

16-03 확인 문제 한 명씩 돌아가며 풀고, 푼 사람 이름을 적어 주세요

1 BLEU는 높을수록 좋은 번역이다. (O / X) 이름 _____

2 BLEU는 뜻이 같은 다른 단어로 번역해도 점수를 준다. (O / X) 이름 _____

3 "the the the the the the the"의 보정된 유니그램 정밀도는? 이름 _____

① 7/7 ② 2/7 ③ 0/7

4 길이 벌점(BP)이 막으려는 것은? 이름 _____

① 너무 짧은 번역이 높은 점수를 받는 것 ② 너무 긴 번역 ③ 같은 단어의 반복

5 BLEU는 1-gram부터 _____-gram까지의 정밀도를 함께 쓴다. 이름 _____

6 네 정밀도를 합칠 때 쓰는 평균은 _____ 평균이다. 이름 _____

7 BLEU가 점수를 내는 방법을 모범 답안에 밑줄 긋기에 빗대어 설명하고, 한계를 하나 말해 보세요. 이름 _____

원문: 16-03 BLEU Score — https://wikidocs.net/217142 (교재의 구현을 짧게 다시 썼고, 우리 번역기의 BLEU를 재는 실습 5와 BLEU의 한계 정리는 이 자료에서 추가했습니다)

오늘의 요약

다시 보기



정의 모음

개념	정의	예시
seq2seq	인코더와 디코더, 두 RNN을 이은 문장 → 문장 모델	I am a student → je suis étudiant
컨텍스트 벡터	인코더의 마지막 은닉 상태(와 셀 상태)	(1, 128, 256)
교사 강요	훈련할 때 디코더에 정답 단어를 입력으로 넣음	<sos> je suis étudiant
세 가지 텐서	인코더 입력, 디코더 입력, 디코더 정답	(N, 8), (N, 16), (N, 16)
보정된 정밀도	정답에 나온 횟수까지만 맞힌 것으로 침	the × 7 → 2/7
BLEU	길이 벌점 × 1~4-gram 정밀도의 기하 평균	Ca1: 0.5046

다음

다음 자료: 17장 어텐션 메커니즘 (10/14 자료 2/2)

오늘의 번역기는 입력 문장 전체를 벡터 하나에 눌러 담았습니다. 문장이 길어지면 앞의 내용이 흐려집니다. 다음 자료에서는 디코더가 단어를 만들 때마다 입력 문장의 모든 단어를 다시 보게 하는 어텐션을 더해, 같은 데이터로 얼마나 나아지는지 비교합니다.

정답과 해설

16-01 시퀀스투시퀀스

1. X — 달라도 됩니다. 디코더는 <eos>가 나올 때까지 필요한 만큼 냅니다.
2. O
3. ②
4. ②
5. <eos>
6. 인코더
7. 예시 답: 한 사람이 문장을 끝까지 듣고 메모 한 장에 요약하면(인코더, 컨텍스트 벡터), 다른 사람이 그 메모만 보고 한 단어씩 말한다(디코더). 메모 한 장에 긴 문장을 다 담기 어려워, 문장이 길수록 내용이 빠진다.

16-02 Seq2Seq를 이용한 번역기 구현하기

1. X
2. X — 마지막 은닉 상태와 셀 상태만 넘깁니다. 모든 시점의 출력을 쓰는 것은 17장의 어텐션입니다.
3. ②
4. ②
5. <sos>
6. 8,087
7. 예시 답: 훈련 때의 정확도는 모범 번역문을 옆에 두고 "여기까지 맞게 썼다면 다음 단어는?"을 채점한 것이다. 실제 번역에서는 자기가 낸 단어가 다음 입력이 되므로, 한 번 틀리면 뒤가 함께 틀어진다.

16-03 BLEU Score

1. O
2. X — 글자가 같은 n-gram만 셉니다.
3. ②
4. ①
5. 4
6. 기하
7. 예시 답: 모범 답안과 겹치는 낱말과 구절에 밑줄을 긋고 그 비율로 점수를 준다. 반복은 한 번만 쳐 주고, 긴 구절이 겹칠수록 높게 치며, 너무 짧은 답은 감점한다. 모범 답안과 다른 말로 쓴 좋은 번역에는 낮은 점수를 준다는 한계가 있다.

용어 정리

용어	정의
시퀀스투시퀀스 (seq2seq)	입력 시퀀스를 받아 다른 시퀀스를 출력하는 인코더-디코더 모델
인코더 / 디코더	입력 문장을 읽는 RNN / 출력 문장을 만드는 RNN
컨텍스트 벡터	입력 문장을 요약한 벡터. 인코더의 마지막 은닉 상태
<sos>, <eos>	문장의 시작과 끝을 알리는 특별 토큰
교사 강요	훈련할 때 디코더의 입력으로 정답 단어를 넣어 주는 방법
병렬 말뭉치	같은 뜻의 문장이 두 언어로 짝지어진 데이터
BLEU	기계 번역과 사람의 번역이 공유하는 n-gram으로 번역의 품질을 재는 점수
Candidate / Reference	기계가 낸 번역 / 사람이 만든 정답 번역
보정된 정밀도 (Count clip)	정답에 나온 횟수까지만 맞힌 것으로 쳐 주는 정밀도
길이 벌점 (BP)	번역이 정답보다 짧을 때 점수를 깎는 값